**Pontificia Universidad Católica Madre y Maestra.**

**Presentado por**

Hiroshy Luna

**Matricula:**

1014-5127

**Asignatura:**

Programacion Web Avanzada

**Dirigido a:**

Carlos Camacho

**Título:**

Practica Docker Simple

# 1. Introducción

El presente informe describe el proceso de contenerización de una aplicación mediante Docker, incluyendo la creación de una imagen optimizada, la configuración de ejecución y su publicación en un repositorio público. El objetivo principal fue aplicar buenas prácticas de construcción de imágenes, optimización de tamaño, persistencia de datos y configuración dinámica, garantizando portabilidad y facilidad de despliegue en distintos entornos.

# 2. Objetivos

## 2.1 Objetivo General

Implementar Docker como plataforma de contenedorización para una aplicación y publicar su imagen en Docker Hub.

## 2.2 Objetivos Específicos

- Construir una imagen Docker optimizada.
- Implementar un proceso de construcción multi-stage.
- Configurar persistencia de logs mediante volúmenes.
- Permitir configuración dinámica de puertos.
- Publicar la imagen en un repositorio remoto.

# 3. Desarrollo de la Práctica

A continuación se describen los componentes técnicos implementados para cumplir con los requisitos de la práctica.

Se utilizó la imagen base `eclipse-temurin:17-jre` en la etapa final del Dockerfile. Esta imagen contiene únicamente el entorno de ejecución de Java (JRE), en lugar del kit de desarrollo completo (JDK), lo cual reduce significativamente el tamaño de la imagen final, situándolo aproximadamente entre 300 MB y 400 MB. Esta optimización mejora el tiempo de descarga, despliegue y consumo de recursos.

Se implementó un proceso de construcción en dos etapas dentro del Dockerfile con el objetivo de separar el entorno de compilación del entorno de ejecución.

**Etapa de Construcción (Builder):**

- Se utilizó la imagen `gradle:8.4-jdk17`.
- Se compiló el código fuente.
- Se generó el archivo ejecutable JAR mediante la tarea `shadowJar`.
- Los archivos temporales y dependencias de compilación se descartaron al finalizar la etapa.

**Etapa Final (Runtime):**

- Se utilizó la imagen `eclipse-temurin:17-jre`.
- Solo se copió el archivo JAR compilado desde la etapa anterior.
- Se obtuvo una imagen final ligera, limpia y adecuada para producci

Para garantizar que los registros de la aplicación no se pierdan al detener o eliminar el contenedor, se configuró un volumen en el directorio `/app/logs`. Esto permite que los logs se almacenen fuera del contenedor y permanezcan disponibles incluso después de reinicios.

Configuración aplicada en el Dockerfile:

```
RUN mkdir -p /app/logs
VOLUME ["/app/logs"]
```

Se modificó la clase principal de la aplicación para permitir que los puertos de ejecución puedan definirse mediante variables de entorno. De esta forma, no es necesario recompilar la imagen cuando se requiere ejecutar el contenedor en puertos distintos.

Código implementado en `Main.java`:

```
int appPort =
Integer.parseInt(System.getenv().getOrDefault("APP_PORT",
"7000"));
int grpcPort =
Integer.parseInt(System.getenv().getOrDefault("GRPC_PORT",
"50051"));
```

Configuración correspondiente en el Dockerfile:

```
ENV APP_PORT=7000
ENV GRPC_PORT=50051
EXPOSE ${APP_PORT}
EXPOSE ${GRPC_PORT}
```

### 3.5 Publicación en Docker Hub

La imagen generada fue publicada exitosamente en un repositorio público de Docker Hub con el fin de permitir su distribución y despliegue desde cualquier entorno compatible con Docker.

**Datos del repositorio:**

- Nombre del repositorio: hbll0001/proyecto-final
- Etiquetas publicadas: latest, 1.0
- Plataforma: Docker Hub

## Evidencia de Funcionamiento

Se ejecutaron los siguientes comandos para validar el proceso completo de construcción, ejecución y publicación de la imagen.

**Construcción de imagen**

```
docker build -t hbll0001/proyecto-final:1.0 .
```

**Ejecución local del contenedor**

```
docker run -d -p 7000:7000 -p 50051:50051 -v LogsVol:/app/logs
hbll0001/proyecto-final:1.0
```

**Publicación en Docker Hub**

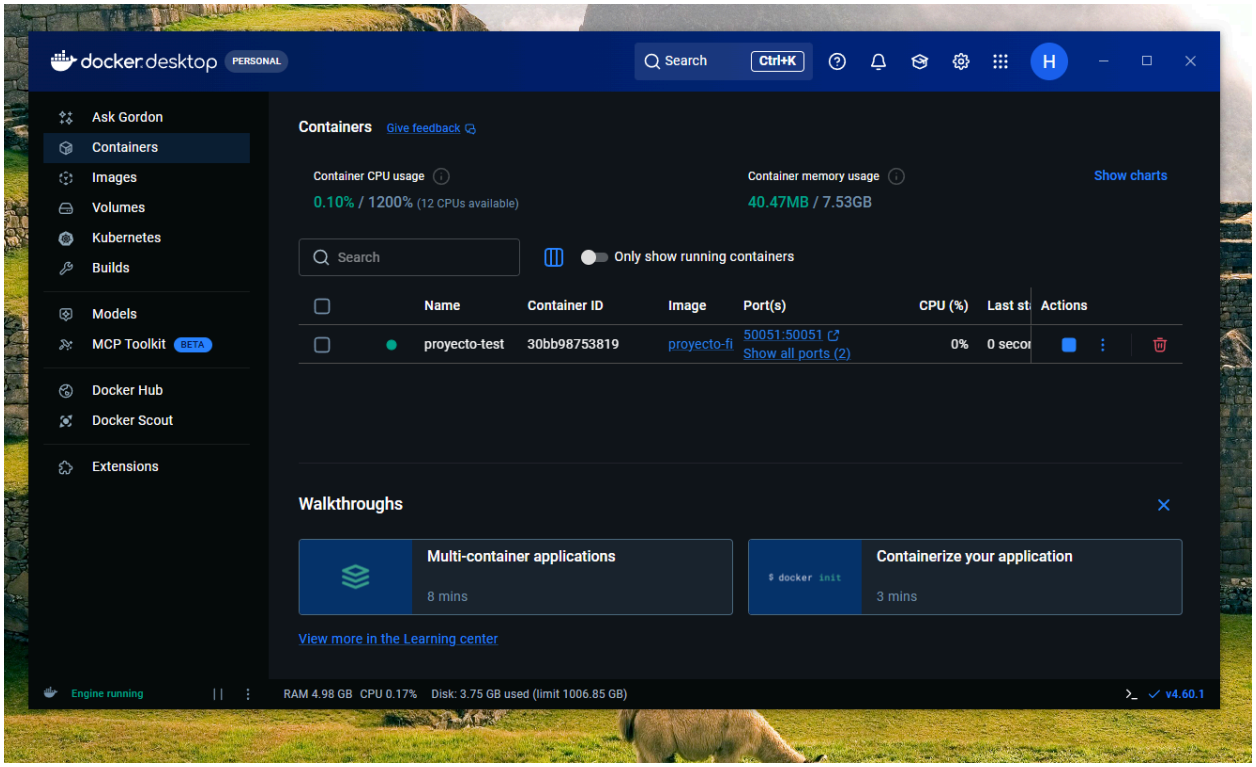
```
docker push hbll0001/proyecto-final:1.0
```

### 4.1 Dockerfile Final Implementado

```
FROM gradle:8.4-jdk17 AS builder
WORKDIR /app
COPY gradlew gradlew.bat settings.gradle ./
COPY gradle ./gradle
COPY app ./app
RUN chmod +x ./gradlew
RUN ./gradlew :app:clean :app:shadowJar --no-daemon

FROM eclipse-temurin:17-jre
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends
ca-certificates curl \
    && rm -rf /var/lib/apt/lists/*
RUN mkdir -p /app/logs
COPY --from=builder /app/app/build/libs/*.jar ./app.jar
ENV APP_PORT=7000
ENV GRPC_PORT=50051
EXPOSE ${APP_PORT}
EXPOSE ${GRPC_PORT}
VOLUME ["/app/logs"]
CMD ["java", "-Djava.net.preferIPv4Stack=true", "-jar",
"app.jar"]
```

Imagenes anexadas como evidencia:



hub

Search Docker Hub Ctrl K Sign in Sign up

Explore / hbll0001 / proyecto-final

## hbll0001/proyecto-final

By hbll0001 · Updated 2 minutes ago

IMAGE

0 0

Overview Tags

Sort by Newest Filter tags

TAG
1.0
Last pushed 2 minutes by hbll0001

```
docker pull hbll0001/proyecto-final:1.0
```

| Digest | OS/ARCH | Compressed size |
|---|---|---|
| 3c71f530702d | linux/amd64 | 133.51 MB |

TAG
latest
Last pushed 2 minutes by hbll0001

```
docker pull hbll0001/proyecto-final:latest
```

| Digest | OS/ARCH | Compressed size |
|---|---|---|
| 3c71f530702d | linux/amd64 | 133.51 MB |

Dockerfile > ...

```
# Instalar certificados raíz y actualizar truststore de Java
RUN apt-get update && apt-get install -y --no-install-recommends ca-certificates curl \
    && update-ca-certificates \
    && keytool -importkeystore \
    -srckeystore /etc/ssl/certs/java/cacerts \
    -destkeystore /etc/ssl/certs/java/cacerts \
    -srcstorepass changeit \
    -deststorepass changeit \
    -noprompt || true \
    && rm -rf /var/lib/apt/lists/*

# Crear directorio para logs
RUN mkdir -p /app/logs

# Copiar el JAR construido
COPY --from=builder /app/app/build/libs/*.jar ./app.jar

# Variables de ambiente con valores por defecto
ENV APP_PORT=7000
ENV GRPC_PORT=50051

# Exponer los puertos de la aplicación
EXPOSE ${APP_PORT}
EXPOSE ${GRPC_PORT}

# Configurar volumen para logs
VOLUME ["/app/logs"]

# Comando para ejecutar el JAR con las variables de entorno
CMD ["java", "-Djava.net.preferIPv4Stack=true", "-jar", "app.jar"]
```

Edit Ctrl+I Chat Ctrl+L ...